

EMG Robotic Hand

Start Here

Build guide: wearable EMG/ECG sensor → Raspberry Pi 5 inference → live web dashboard & force-aware 5-finger robotic hand. With 3 fingertip FSRs for adaptive grip.

A wearable EMG → 5-finger robotic hand with **force-aware grip** (won't crush an egg) plus a live ECG / vital-signs dashboard. One ESP32 streams biosignals + fingertip force readings wirelessly to a Pi 5; the Pi runs gesture inference, drives the servos, and serves a real-time dashboard.

Folder layout

```
emg_hand/
|-- START_HERE.md / .tex / .pdf      (this guide)
|-- EMG_Hand_Presentation.pptx      8-slide deck for the demo
|-- build_presentation.js            pptxgenjs source - `node build_presentation.js` rebuilds .pptx
|
|-- firmware/
|   |-- esp32_emg_streamer/
|   |   |-- esp32_emg_streamer.ino    2-ch EMG + 3-FSR streaming firmware (54-byte UDP packets)
|   |
|   |-- pi/
|   |   |-- emg_common.py            everything that runs on the Pi
|   |   |-- ecg_processing.py        packet parser, EMG filter, 1D-CNN, normaliser
|   |   |-- record_gestures.py       R-peak detection, HR, HRV, respiration
|   |   |-- train_gestures.py        record labelled EMG gesture data
|   |   |-- predict_realtime.py      train the 1D-CNN classifier
|   |   |-- dashboard_server.py      live inference + force-aware HandController
|   |   |-- dashboard.html           FastAPI + WebSocket - runs ECG + EMG + gesture inference
|   |   |-- servo_calibrate.py       browser UI (current-gesture, confidence bars, force gauges,
|   |                                   EMG waveform, ECG, HR/HRV/RR tachogram)
|   |                                   interactive servo calibration REPL
|   |
|   |-- docs/
|   |   |-- parts_and_setup.html      open in any browser
|   |   |-- electronics_setup.html    BOM + physical layout
|   |   |-- pi_servo_wiring.html      full pin-level wiring
|   |   |-- wearable_design.html     Pi 5 <-> PCA9685 <-> servos
|   |                                   3D-printable forearm pod design
```

What it does

- **Wearable** (forearm-mounted): ESP32 + 2× AD8232 + 3 fingertip FSR-402 force sensors. Streams 2-ch EMG at 1 kHz *and* 3-ch fingertip force at 100 Hz over WiFi UDP in 54-byte packets.
- **Pi 5** receives that stream and runs:
 - **Force-aware gesture control** — 1D CNN classifies the EMG window into one of five gestures (rest / fist / open / pinch / point), then the HandController steps each servo toward the target pose. **A finger stops closing the moment its FSR reading exceeds the per-finger threshold** — that's the egg-grip behaviour.
 - **Live web dashboard** at <http://<pi-ip>:8000/> showing current gesture, classifier confidence bars, fingertip force gauges, 2-channel EMG waveform, plus ECG / HR / HRV / respiration / RR tachogram.
- The dashboard server and `predict_realtime.py` both bind UDP :5555 — only one at a time per ESP32.

Quick start

Step 0 — Read the visual references

Open these in a browser before wiring (double-click the .html):

1. docs/parts_and_setup.html — BOM (6 component types, 11 units, ~£75 to source).
2. docs/electronics_setup.html — pin-level wiring of ESP32 + AD8232.

3. docs/pi_servo_wiring.html — Pi 5 to PCA9685 to servos.
4. docs/wearable_design.html — 3D-printable forearm pod (optional).
5. EMG_Hand_Presentation.pptx — the demo deck (8 slides, 5-min slot).
6. demo_video_script.md — 60-second demo-video script + shot list.

Step 1 — Flash the ESP32

1. Install Arduino IDE 2.x + ESP32 board support (Boards Manager → esp32).
2. Open firmware/esp32_emg_streamer/esp32_emg_streamer.ino.
3. Edit the network details at the top:

```
const char* WIFI_SSID    = "<your-network>";
const char* WIFI_PASSWORD = "<your-password>";
const char* PI_IP        = "<your-pi-ip>";    // `hostname -I` on the Pi
```

4. If FSRs aren't physically wired yet, also set FSR_ENABLED = false (sends zeros, doesn't crash).
5. Select board **ESP32 Dev Module** + correct port, upload.
6. Open Serial Monitor at **115200 baud**. Expect:

```
UDP ready -> <pi-ip>:5555
[5s] Pkts:498 | Smp:5000 | Rate:1000Hz | L01:OK L02:OK | FSR T/I/M:  45/  38/  52 | WiFi:OK
(-52)
```

Step 2 — Wire the analog frontend + FSRs

Follow docs/electronics_setup.html Panel B2.

		AD8232 #1 (CH1)	AD8232 #2 (CH2)
AD8232 modules → ESP32	3.3V	→ 3V3	3.3V → 3V3
	GND	→ GND	GND → GND
	OUTPUT	→ GPIO 34	OUTPUT → GPIO 35
	L0+	→ GPIO 25	L0+ → GPIO 26
	L0−	→ GPIO 27	L0− <i>unused (pin freed)</i>

	Sensor index	ADC pin	Channel notes
Fingertip FSR-402 force sensors (voltage divider to GND)	FSR thumb (<code>fsr[0]</code>)	GPIO 33	ADC1_CH5
	FSR index (<code>fsr[1]</code>)	GPIO 36	ADC1_CH0 (input-only)
	FSR middle (<code>fsr[2]</code>)	GPIO 39	ADC1_CH3 (input-only)

Per FSR: 3V3 → FSR → ADC pin → 10 kΩ → GND. No force → ADC ≈ 0. Hard squeeze → ADC ≥ 3500.

Both AD8232 RL pins go to a single shared reference electrode (bony spot — olecranon or ulnar styloid).

Step 3 — Set up the Pi

```
# Copy this project onto the Pi at ~/Documents/emg_hand
cd ~/Documents/emg_hand/pi

# Use a venv (Pi OS Bookworm is externally-managed)
python3 -m venv ~/emg-venv
source ~/emg-venv/bin/activate
pip install -U pip
pip install numpy scipy torch scikit-learn fastapi 'uvicorn[standard]'

# Only needed if you'll drive the robotic hand:
sudo raspi-config      # Interface Options > I2C > Enable, then reboot
pip install adafruit-circuitpython-servokit
```

Step 4 — Verify the data link

With the ESP32 powered and electrodes attached:

```
# On the Pi
sudo tcpdump -i any -n udp port 5555 -c 5
```

You should see 5 packets received within a second — each is **54 bytes**. If silent, `PI_IP` in the `.ino` is wrong, or your network isolates clients (most university / hotel guest WiFi does — use a phone hotspot).

Step 5 — Pick a mode

Mode A — Live ECG / vital-signs dashboard

1. **Re-wire AD8232 #1 to chest** (Lead I): RA → upper-right chest, LA → upper-left, RL → right hip.
2. **Run the server:**

```
source ~/emg-venv/bin/activate
cd ~/Documents/emg_hand/pi
python dashboard_server.py
```

3. **Open** `http://<pi-ip>:8000/` in any browser. Within ~10s of stable contact you should see HR, HRV, the ECG waveform, and the RR-interval tachogram updating live.

Mode B — Force-aware gesture control + robotic hand

This is the demo path. Six small stages.

Stage 1 — electrodes on forearm. CH1 pair over flexor digitorum (palmar, ~1/3 down from elbow), CH2 pair over extensor digitorum (dorsal), shared reference on the olecranon.

```
cd ~/Documents/emg_hand/pi
python record_gestures.py --reps 3 --duration 5
```

Walks through each gesture for $5\text{ s} \times 3\text{ reps}$. Saves `gestures.npz`.

```
python train_gestures.py --epochs 80
```

Prints test accuracy + confusion matrix. Saves `emg_model.pth` + `emg_norm.npz`.

Stage 4 — glue FSRs to fingertips, set thresholds. Wire the FSRs (Step 2). Run `servo_calibrate.py` while squeezing each FSR pad to find raw readings at rest (~ 50) and at hard squeeze (~ 3500). Open `predict_realtime.py` and tune:

```
FORCE_THRESHOLD = [1800, 1800, 1800] # thumb, index, middle
```

Match the same numbers in `dashboard_server.py` `FORCE_THRESHOLDS` so the gauge tick lines up.

```
python servo_calibrate.py
```

Interactive REPL — find the angles for “fully open” / “fully closed” per finger, save snapshots, then paste the result dict into `HandController.GESTURE_ANGLES` in `predict_realtime.py`.

```
python predict_realtime.py
```

Without a PCA9685 connected: prints gesture + servo angles to terminal (force-blocking events printed too).
With one connected: drives the servo hand. The closing direction is force-blocked per finger.

servo_calibrate.py quick reference

```
> 1 45          # set channel 1 (index) to 45 deg
> s 0           # sweep channel 0 (thumb) 0 -> 180 -> 0
> s all         # sweep all 5 sequentially
> open          # all servos -> 180 deg
> close         # all servos -> 0 deg
> rest         # all servos -> 90 deg (safe centre)
> snap fist     # save current pose as 'fist'
> snap pinch    # save current pose as 'pinch'
> show          # prints GESTURE_ANGLES dict to paste into predict_realtime.py
> q            # quit (leaves servos where they are)
```

Demo prep — slide deck + video

Stage 6 — run live inference.

- `EMG_Hand_Presentation.pptx` — 8-slide deck sized for a 5-minute slot. Two things to fill in: a photo of the wearable + hand (slide 4), and your real confusion-matrix numbers (slide 7). To rebuild after edits: `node build_presentation.js`.
- `demo_video_script.md` — 60-second demo-video script + 26-shot list + equipment checklist + editing notes. The signature sequence is Beat 3 (egg-grip moment), 28 seconds out of 60. Plan for ≥ 3 takes of the egg pickup.

Why the egg-grip is the headline: most low-cost robotic hands either crush things or aren't strong enough. Yours stops at exactly the right moment, and the dashboard makes the closed loop *visible* — the audience watches the force gauge cross the threshold tick, sees the bar turn red, and sees the hand halt. That single visual is worth more than any number on a results slide.

Troubleshooting

Symptom	Likely cause
ESP32 stuck on <code>Connecting...</code>	WiFi credentials wrong, or hotspot not broadcasting. Check from your phone.
<code>tcpdump</code> silent on the Pi	PI_IP wrong, or guest network isolates clients. Use a phone hotspot.
Flat / dead EMG signal	~90% of the time it's electrode contact. Clean skin with alcohol, abrade lightly, re-stick. Orange LO LED on AD8232 confirms a lead is off.
FSR readings stuck near 0 with no force applied	Voltage divider not built, or 10 k Ω resistor missing. ADC reads a floating pin as noise around 0 / drifting.
FSR readings stuck near 4095 (saturated)	FSR shorted, or wired without divider. The ADC sees 3.3 V directly.
Hand crushes the egg every take	Lower <code>FORCE_THRESHOLD</code> by 200 in <code>predict_realtime.py</code> ; recalibrate.
Hand stops too early, never grasps	Raise <code>FORCE_THRESHOLD</code> ; check the FSR is actually contacting at full close.
Dashboard shows -- for HR forever	Need ≥ 2 R-peaks. Improve electrode placement, lower <code>RMS_GATE</code> , or wait 30 s for filter warm-up.
Gesture panel says “no model loaded”	Run <code>record_gestures.py</code> + <code>train_gestures.py</code> first; <code>emg_model.pth</code> + <code>emg_norm.npz</code> must be next to <code>dashboard_server.py</code> .
Gesture model collapses to one class	StandardScaler leakage or class imbalance. Re-record with longer rest and more reps, then retrain.
Servos jitter / don't respond	Forgot to tie PSU GND to Pi GND — the most common build mistake. See <code>pi_servo_wiring.html</code> .
<code>i2cdetect -y 1</code> doesn't show 0x40	I ² C not enabled (<code>raspi-config</code>), SDA/SCL swapped, or PCA9685 missing its 3.3 V supply.
<code>predict_realtime.py</code> “print-only mode”	<code>adafruit-servokit</code> not installed in the active venv. <code>pip install adafruit-circuitpython-servokit</code> .

Single most common build mistake: servo PSU ground not tied to Pi ground. Without a common ground the PWM logic levels float and servos either don't move or twitch unpredictably.

What's next

If the demo lands and you want to keep going past the competition, the realistic upgrades — each modular, each ≤ 1 weekend:

- **MAX30102** (~£5, I²C) — SpO₂ + PPG + optical HR cross-check. Add a dashboard panel.
- **MPU6050** (~£3, I²C) — wrist orientation, activity classification, gesture disambiguation.
- **NTC thermistor** (~£1) — skin temperature trend.
- **Custom PCB** — consolidates ESP32 + 2 $\times$ AD8232 onto one board, eliminates the breadboard, looks like a product.
- **Pre-train on Ninapro DB2**, fine-tune on your own data — cross-subject robustness.
- **1D Transformer comparison** — adds an architectural ablation to your results slide. Likely loses to the CNN at this data scale, which is itself a finding.

For *real* EEG you'd need an ADS1299-class front-end (OpenBCI Ganglion or Cyton). The AD8232 isn't built for scalp-microvolt recordings.